Guía Completa de Rust: De Principiante a Experto

Tabla de Contenidos

29. Procedural Macros
30. Embedded Rust
31. WebAssembly
32. Ecosistema y Crates Importantes

NIVEL PRINCIPIANTE

1. Introducción a Rust

Rust es un lenguaje de programación de sistemas que se enfoca en: - **Seguridad de memoria**: Previene errores comunes como buffer overflows - **Concurrencia sin data races**: Garantiza thread safety en tiempo de compilación - **Performance**: Velocidad comparable a C/C++ - **Zero-cost abstractions**: Las abstracciones no tienen overhead en runtime

¿Por qué Rust? - Elimina crashes por acceso a memoria inválida - Sin garbage collector, pero con gestión automática de memoria - Compilador extremadamente útil y descriptivo - Ecosistema moderno y creciente

2. Instalación y Configuración

Instalar Rust:

```
# Instalar rustup (gestor de versiones de Rust)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Actualizar
rustup update

# Verificar instalación
rustc --version
cargo --version
```

Configurar Editor: - VS Code: Extensión "rust-analyzer" - IntelliJ: Plugin Rust - Vim/Neovim: rust.vim + coc-rust-analyzer

3. Primeros Pasos

Crear un proyecto:

```
cargo new mi_proyecto
cd mi_proyecto
cargo run
```

Estructura del proyecto:

```
mi_proyecto/
  Cargo.toml    # Configuración y dependencias
```

```
  src/
     main.rs   # Código fuente
  target/       # Archivos compilados
```

Hello World:

```rust
fn main() {
    println!("¡Hola, mundo!");
}
```

Comandos básicos de Cargo:

```
cargo new proyecto      # Crear nuevo proyecto
cargo build             # Compilar
cargo run               # Compilar y ejecutar
cargo check             # Verificar sin compilar
cargo test              # Ejecutar tests
cargo doc               # Generar documentación
```

4. Variables y Mutabilidad

Variables inmutables (por defecto):

```rust
fn main() {
    let x = 5;
    println!("El valor de x es: {}", x);
    // x = 6; // ¡Error! No se puede reasignar
}
```

Variables mutables:

```rust
fn main() {
    let mut x = 5;
    println!("El valor de x es: {}", x);
    x = 6;
    println!("El valor de x es: {}", x);
}
```

Shadowing:

```rust
fn main() {
    let x = 5;
    let x = x + 1;  // Nueva variable, mismo nombre
    let x = x * 2;  // Otra nueva variable
    println!("El valor de x es: {}", x); // 12

    // Cambiar tipo con shadowing
    let spaces = "   ";
    let spaces = spaces.len(); // String -> número
}
```

Constantes:

```
const THREE_HOURS_IN_SECONDS: u32 = 60 * 60 * 3;

fn main() {
    println!("Tres horas son {} segundos", THREE_HOURS_IN_SECONDS);
}
```

5. Tipos de Datos

Tipos escalares:

Enteros:

```
fn main() {
    let a: i8 = -128;
    let b: u8 = 255;
    let c: i32 = -2_147_483_648;
    let d: u32 = 4_294_967_295;
    let e: isize = -9223372036854775808;
}
```

15. Módulos y Crates

Estructura de módulos:

```
// src/lib.rs
mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}

        fn seat_at_table() {}
    }

    mod serving {
        fn take_order() {}

        fn serve_order() {}

        fn take_payment() {}
    }
}

pub fn eat_at_restaurant() {
    // Ruta absoluta
    crate::front_of_house::hosting::add_to_waitlist();

    // Ruta relativa
```

```
    front_of_house::hosting::add_to_waitlist();
}
```

super keyword:

```
fn deliver_order() {}

mod back_of_house {
    fn fix_incorrect_order() {
        cook_order();
        super::deliver_order(); // Referencia al padre
    }

    fn cook_order() {}

    pub struct Breakfast {
        pub toast: String,
        seasonal_fruit: String, // Privado
    }

    impl Breakfast {
        pub fn summer(toast: &str) -> Breakfast {
            Breakfast {
                toast: String::from(toast),
                seasonal_fruit: String::from("peaches"),
            }
        }
    }

    pub enum Appetizer {
        Soup,    // Público automáticamente
        Salad,   // Público automáticamente
    }
}

pub fn eat_at_restaurant() {
    let mut meal = back_of_house::Breakfast::summer("Rye");
    meal.toast = String::from("Wheat");
    println!("I'd like {} toast please", meal.toast);

    // meal.seasonal_fruit = String::from("blueberries"); // ¡Error!

    let order1 = back_of_house::Appetizer::Soup;
    let order2 = back_of_house::Appetizer::Salad;
}
```

use keyword:

```rust
mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}
    }
}

use crate::front_of_house::hosting;

pub fn eat_at_restaurant() {
    hosting::add_to_waitlist();
}

// Alias
use std::fmt::Result;
use std::io::Result as IoResult;

// Re-exporting
pub use crate::front_of_house::hosting;

// use con glob
use std::collections::*;
```

Separar módulos en archivos:

```
src/
  main.rs
  lib.rs
  front_of_house.rs
  front_of_house/
      hosting.rs
      serving.rs
```

```rust
// src/lib.rs
mod front_of_house;

pub use crate::front_of_house::hosting;

pub fn eat_at_restaurant() {
    hosting::add_to_waitlist();
}

// src/front_of_house.rs
pub mod hosting;

// src/front_of_house/hosting.rs
pub fn add_to_waitlist() {}
```

Cargo workspace:

```toml
# Cargo.toml (raíz)
[workspace]
members = [
    "adder",
    "add_one",
]

# adder/Cargo.toml
[dependencies]
add_one = { path = "../add_one" }

# add_one/Cargo.toml
[package]
name = "add_one"
version = "0.1.0"
edition = "2021"
```

NIVEL INTERMEDIO-AVANZADO

16. Generics

Funciones genéricas:

```rust
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];
    let result = largest(&number_list);
    println!("El número más grande es {}", result);

    let char_list = vec!['y', 'm', 'a', 'q'];
    let result = largest(&char_list);
    println!("El char más grande es {}", result);
}
```

Structs genéricos:

```rust
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

// Implementación específica para f32
impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}

// Múltiples tipos genéricos
struct Point2<T, U> {
    x: T,
    y: U,
}

impl<T, U> Point2<T, U> {
    fn mixup<V, W>(self, other: Point2<V, W>) -> Point2<T, W> {
        Point2 {
            x: self.x,
            y: other.y,
        }
    }
}

fn main() {
    let integer = Point { x: 5, y: 10 };
    let float = Point { x: 1.0, y: 4.0 };

    let both_integer = Point2 { x: 5, y: 10 };
    let both_float = Point2 { x: 1.0, y: 4.0 };
    let integer_and_float = Point2 { x: 5, y: 4.0 };

    let p1 = Point2 { x: 5, y: 10.4 };
    let p2 = Point2 { x: "Hello", y: 'c' };
    let p3 = p1.mixup(p2);
    println!("p3.x = {}, p3.y = {}", p3.x, p3.y);
}
```

Enums genéricos:

```rust
enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

17. Traits

Definir un trait:

```rust
pub trait Summary {
    fn summarize(&self) -> String;

    // Implementación por defecto
    fn summarize_author(&self) -> String {
        String::from("(Read more...)")
    }
}

pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("{}, by {} ({})", self.headline, self.author, self.location)
    }
}

pub struct Tweet {
    pub username: String,
    pub content: String,
    pub reply: bool,
    pub retweet: bool,
}

impl Summary for Tweet {
    fn summarize(&self) -> String {
```

```
        format!("{}: {}", self.username, self.content)
    }
}

fn main() {
    let tweet = Tweet {
        username: String::from("horse_ebooks"),
        content: String::from("of course, as you probably already know, people"),
        reply: false,
        retweet: false,
    };

    println!("1 new tweet: {}", tweet.summarize());
}
```

Traits como parámetros:

```
pub fn notify(item: &impl Summary) {
    println!("Breaking news! {}", item.summarize());
}

// Trait bound syntax
pub fn notify2<T: Summary>(item: &T) {
    println!("Breaking news! {}", item.summarize());
}

// Múltiples trait bounds
pub fn notify3(item: &(impl Summary + Display)) {
    // ...
}

pub fn notify4<T: Summary + Display>(item: &T) {
    // ...
}

// where clauses
fn some_function<T, U>(t: &T, u: &U) -> i32
where
    T: Display + Clone,
    U: Clone + Debug,
{
    // ...
}
```

Retornar tipos que implementan traits:

```
fn returns_summarizable() -> impl Summary {
    Tweet {
```

```
        username: String::from("horse_ebooks"),
        content: String::from("of course, as you probably already know, people"),
        reply: false,
        retweet: false,
    }
}
```

Implementaciones condicionales:

```
use std::fmt::Display;

struct Pair<T> {
    x: T,
    y: T,
}

impl<T> Pair<T> {
    fn new(x: T, y: T) -> Self {
        Self { x, y }
    }
}

impl<T: Display + PartialOrd> Pair<T> {
    fn cmp_display(&self) {
        if self.x >= self.y {
            println!("The largest member is x = {}", self.x);
        } else {
            println!("The largest member is y = {}", self.y);
        }
    }
}
```

Blanket implementations:

```
// Implementación para cualquier tipo que implemente Display
impl<T: Display> ToString for T {
    // ...
}
```

Traits importantes del standard library:

```
#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord)]
struct Person {
    name: String,
    age: u32,
}

impl std::fmt::Display for Person {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
```

```
        write!(f, "{} ({} años)", self.name, self.age)
    }
}

fn main() {
    let person1 = Person {
        name: String::from("Alice"),
        age: 30,
    };

    let person2 = person1.clone();

    println!("{}", person1); // Display
    println!("{:?}", person1); // Debug
    println!("{:#?}", person1); // Pretty Debug

    println!("Son iguales: {}", person1 == person2); // PartialEq
}
```

18. Lifetimes

El problema que resuelven los lifetimes:

```
fn main() {
    let r;

    {
        let x = 5;
        r = &x;  // x no vive lo suficiente
    }

    // println!("r: {}", r); // ¡Error!
}
```

Lifetimes en funciones:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let string1 = String::from("abcd");
    let string2 = "xyz";
```

```
    let result = longest(string1.as_str(), string2);
    println!("The longest string is {}", result);
}
```

Lifetime annotation syntax:

```
&i32        // una referencia
&'a i32     // una referencia con lifetime explícito
&'a mut i32 // una referencia mutable con lifetime explícito
```

Lifetimes en structs:

```
struct ImportantExcerpt<'a> {
    part: &'a str,
}

impl<'a> ImportantExcerpt<'a> {
    fn level(&self) -> i32 {
        3
    }

    fn announce_and_return_part(&self, announcement: &str) -> &str {
        println!("Attention please: {}", announcement);
        self.part
    }
}

fn main() {
    let novel = String::from("Call me Ishmael. Some years ago...");
    let first_sentence = novel.split('.').next().expect("Could not find a '.'");
    let i = ImportantExcerpt {
        part: first_sentence,
    };
}
```

Lifetime elision rules:

```
// Regla 1: Cada parámetro que es referencia obtiene su propio lifetime
fn first_word(s: &str) -> &str { // se convierte en:
// fn first_word<'a>(s: &'a str) -> &str {

// Regla 2: Si hay exactamente un lifetime input, se asigna al output
fn first_word<'a>(s: &'a str) -> &'a str {

// Regla 3: Si hay &self o &mut self, su lifetime se asigna al output
impl<'a> ImportantExcerpt<'a> {
    fn level(&self) -> i32 { // No necesita anotaciones
```

```
    }
}
```

Static lifetime:

```
fn main() {
    let s: &'static str = "I have a static lifetime.";
    // String literals siempre tienen 'static lifetime
}
```

Generic type parameters, trait bounds, y lifetimes juntos:

```
use std::fmt::Display;

fn longest_with_an_announcement<'a, T>(
    x: &'a str,
    y: &'a str,
    ann: T,
) -> &'a str
where
    T: Display,
{
    println!("Announcement! {}", ann);
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

19. Testing

Tests básicos:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
    }

    #[test]
    fn another() {
        panic!("Make this test fail");
    }
```

```rust
    #[test]
    fn larger_can_hold_smaller() {
        let larger = Rectangle {
            width: 8,
            height: 7,
        };
        let smaller = Rectangle {
            width: 5,
            height: 1,
        };

        assert!(larger.can_hold(&smaller));
    }

    #[test]
    fn smaller_cannot_hold_larger() {
        let larger = Rectangle {
            width: 8,
            height: 7,
        };
        let smaller = Rectangle {
            width: 5,
            height: 1,
        };

        assert!(!smaller.can_hold(&larger));
    }
}

pub fn add(left: usize, right: usize) -> usize {
    left + right
}

struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}
```

Macros de testing:

```rust
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn greeting_contains_name() {
        let result = greeting("Carol");
        assert!(
            result.contains("Carol"),
            "Greeting did not contain name, value was `{}`",
            result
        );
    }

    #[test]
    #[should_panic]
    fn greater_than_100() {
        Guess::new(200);
    }

    #[test]
    #[should_panic(expected = "less than or equal to 100")]
    fn greater_than_100_with_message() {
        Guess::new(200);
    }

    #[test]
    fn it_works() -> Result<(), String> {
        if 2 + 2 == 4 {
            Ok(())
        } else {
            Err(String::from("two plus two does not equal four"))
        }
    }
}

pub fn greeting(name: &str) -> String {
    format!("Hello {}!", name)
}

pub struct Guess {
    value: i32,
}

impl Guess {
    pub fn new(value: i32) -> Guess {
```

```
        if value < 1 {
            panic!("Guess value must be greater than or equal to 1, got {}.", value);
        } else if value > 100 {
            panic!("Guess value must be less than or equal to 100, got {}.", value);
        }

        Guess { value }
    }
}
```

Controlando cómo se ejecutan los tests:

```
# Ejecutar tests en paralelo (por defecto)
cargo test

# Ejecutar tests de forma secuencial
cargo test -- --test-threads=1

# Mostrar output de println! en tests exitosos
cargo test -- --show-output

# Ejecutar un test específico
cargo test it_works

# Ejecutar tests que contengan una palabra
cargo test add

# Ignorar tests costosos
cargo test -- --ignored
```

Tests de integración:

```
// tests/integration_test.rs
use adder;

#[test]
fn it_adds_two() {
    assert_eq!(4, adder::add_two(2));
}

// tests/common/mod.rs
pub fn setup() {
    // setup code specific to your library's tests would go here
}

// tests/integration_test.rs
use adder;
```

```rust
mod common;

#[test]
fn it_adds_two() {
    common::setup();
    assert_eq!(4, adder::add_two(2));
}
```

20. Iteradores y Closures

Closures:

```rust
use std::thread;
use std::time::Duration;

fn main() {
    let expensive_closure = |num| {
        println!("calculating slowly...");
        thread::sleep(Duration::from_secs(2));
        num
    };

    // Inferencia de tipos
    let add_one_v1 = |x: u32| -> u32 { x + 1 };
    let add_one_v2 = |x|             { x + 1 };
    let add_one_v3 = |x|               x + 1  ;

    // Capturar el entorno
    let x = 4;
    let equal_to_x = |z| z == x;
    let y = 4;
    assert!(equal_to_x(y));
}
```

Closure traits:

```rust
fn main() {
    // FnOnce - toma ownership
    let x = vec![1, 2, 3];
    let equal_to_x = move |z| z == x;
    // println!("{:?}", x); // Error! x fue movido

    // FnMut - mutable borrow
    let mut x = vec![1, 2, 3];
    let mut push_to_x = |item| x.push(item);
    push_to_x(4);
```

```rust
    // Fn - immutable borrow
    let x = vec![1, 2, 3];
    let print_x = || println!("{:?}", x);
    print_x();
    println!("{:?}", x); // x sigue disponible
}
```

Iteradores:

```rust
fn main() {
    let v1 = vec![1, 2, 3];

    // iter() - referencias inmutables
    let v1_iter = v1.iter();
    for val in v1_iter {
        println!("Got: {}", val);
    }

    // into_iter() - toma ownership
    let v1 = vec![1, 2, 3];
    let v1_iter = v1.into_iter();
    for val in v1_iter {
        println!("Got: {}", val);
    }

    // iter_mut() - referencias mutables
    let mut v1 = vec![1, 2, 3];
    let v1_iter = v1.iter_mut();
    for val in v1_iter {
        *val += 1;
    }
}
```

Iterator trait:

```rust
pub trait Iterator {
    type Item;

    fn next(&mut self) -> Option<Self::Item>;

    // métodos con implementación por defecto...
}

fn main() {
    let v1 = vec![1, 2, 3];
    let mut v1_iter = v1.iter();

    assert_eq!(v1_iter.next(), Some(&1));
```

```
    assert_eq!(v1_iter.next(), Some(&2));
    assert_eq!(v1_iter.next(), Some(&3));
    assert_eq!(v1_iter.next(), None);
}
```

Iterator adaptors:

```
fn main() {
    let v1: Vec<i32> = vec![1, 2, 3];

    // map es lazy - necesita collect para ejecutar
    let v2: Vec<_> = v1.iter().map(|x| x + 1).collect();
    assert_eq!(v2, vec![2, 3, 4]);

    // filter
    let shoes = vec![
        Shoe { size: 10, style: String::from("sneaker") },
        Shoe { size: 13, style: String::from("sandal") },
        Shoe { size: 10, style: String::from("boot") },
    ];

    let in_my_size = shoes_in_size(shoes, 10);
}

#[derive(PartialEq, Debug)]
struct Shoe {
    size: u32,
    style: String,
}

fn shoes_in_size(shoes: Vec<Shoe>, shoe_size: u32) -> Vec<Shoe> {
    shoes.into_iter().filter(|s| s.size == shoe_size).collect()
}
```

Crear iterador personalizado:

```
struct Counter {
    current: usize,
    max: usize,
}

impl Counter {
    fn new(max: usize) -> Counter {
        Counter { current: 0, max }
    }
}

impl Iterator for Counter {
```

```rust
    type Item = usize;

    fn next(&mut self) -> Option<Self::Item> {
        if self.current < self.max {
            let current = self.current;
            self.current += 1;
            Some(current)
        } else {
            None
        }
    }
}

fn main() {
    let mut counter = Counter::new(5);

    // Usar directamente
    assert_eq!(counter.next(), Some(0));
    assert_eq!(counter.next(), Some(1));

    // Usar con otros métodos de iterator
    let sum: usize = Counter::new(5)
        .zip(Counter::new(5).skip(1))
        .map(|(a, b)| a * b)
        .filter(|x| x % 3 == 0)
        .sum();

    assert_eq!(18, sum);
}
```

21. Smart Pointers

Box:

```rust
fn main() {
    // Almacenar datos en heap
    let b = Box::new(5);
    println!("b = {}", b);

    // Recursive types
    let list = Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))));
}

enum List {
    Cons(i32, Box<List>),
    Nil,
```

```
}

use crate::List::{Cons, Nil};
```

Deref trait:

```
use std::ops::Deref;

struct MyBox<T>(T);

impl<T> MyBox<T> {
    fn new(x: T) -> MyBox<T> {
        MyBox(x)
    }
}

impl<T> Deref for MyBox<T> {
    type Target = T;

    fn deref(&self) -> &Self::Target {
        &self.0
    }
}

fn main() {
    let x = 5;
    let y = MyBox::new(x);

    assert_eq!(5, x);
    assert_eq!(5, *y); // Equivalente a *(y.deref())

    // Deref coercion
    let m = MyBox::new(String::from("Rust"));
    hello(&m); // &MyBox<String> -> &String -> &str
}

fn hello(name: &str) {
    println!("Hello, {}!", name);
}
```

Drop trait:

```
struct CustomSmartPointer {
    data: String,
}

impl Drop for CustomSmartPointer {
    fn drop(&mut self) {
```

```
        println!("Dropping CustomSmartPointer with data `{}`!", self.data);
    }
}

fn main() {
    let c = CustomSmartPointer {
        data: String::from("my stuff"),
    };
    let d = CustomSmartPointer {
        data: String::from("other stuff"),
    };
    println!("CustomSmartPointers created.");

    // Forzar drop temprano
    drop(c);
    println!("CustomSmartPointer dropped before the end of main.");
}
```

Rc (Reference Counted):

```
use std::rc::Rc;

enum List {
    Cons(i32, Rc<List>),
    Nil,
}

use crate::List::{Cons, Nil};

fn main() {
    let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))));
    println!("count after creating a = {}", Rc::strong_count(&a));

    let b = Cons(3, Rc::clone(&a));
    println!("count after creating b = {}", Rc::strong_count(&a));

    {
        let c = Cons(4, Rc::clone(&a));
        println!("count after creating c = {}", Rc::strong_count(&a));
    }

    println!("count after c goes out of scope = {}", Rc::strong_count(&a));
}
```

RefCell y interior mutability:

```
use std::cell::RefCell;
```

```rust
#[derive(Debug)]
enum List {
    Cons(Rc<RefCell<i32>>, Rc<List>),
    Nil,
}

use crate::List::{Cons, Nil};
use std::rc::Rc;

fn main() {
    let value = Rc::new(RefCell::new(5));

    let a = Rc::new(Cons(Rc::clone(&value), Rc::new(Nil)));

    let b = Cons(Rc::new(RefCell::new(3)), Rc::clone(&a));
    let c = Cons(Rc::new(RefCell::new(4)), Rc::clone(&a));

    *value.borrow_mut() += 10;

    println!("a after = {:?}", a);
    println!("b after = {:?}", b);
    println!("c after = {:?}", c);
}

// Mock object con RefCell
pub trait Messenger {
    fn send(&self, msg: &str);
}

pub struct LimitTracker<'a, T: Messenger> {
    messenger: &'a T,
    value: usize,
    max: usize,
}

impl<'a, T> LimitTracker<'a, T>
where
    T: Messenger,
{
    pub fn new(messenger: &'a T, max: usize) -> LimitTracker<'a, T> {
        LimitTracker {
            messenger,
            value: 0,
            max,
        }
    }
```

```
    pub fn set_value(&mut self, value: usize) {
        self.value = value;

        let percentage_of_max = self.value as f64 / self.max as f64;

        if percentage_of_max >= 1.0 {
            self.messenger.send("Error: You are over your quota!");
        } else if percentage_of_max >= 0.9 {
            self.messenger
                .send("Urgent warning: You've used up over 90% of your quota!");
        } else if percentage_of_max >= 0.75 {
            self.messenger
                .send("Warning: You've used up over 75% of your quota!");
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::cell::RefCell;

    struct MockMessenger {
        sent_messages: RefCell<Vec<String>>,
    }

    impl MockMessenger {
        fn new() -> MockMessenger {
            MockMessenger {
                sent_messages: RefCell::new(vec![]),
            }
        }
    }

    impl Messenger for MockMessenger {
        fn send(&self, message: &str) {
            self.sent_messages.borrow_mut().push(String::from(message));
        }
    }

    #[test]
    fn it_sends_an_over_75_percent_warning_message() {
        let mock_messenger = MockMessenger::new();
        let mut limit_tracker = LimitTracker::new(&mock_messenger, 100);
```

```
        limit_tracker.set_value(80);

        assert_eq!(mock_messenger.sent_messages.borrow().len(), 1);
    }
}
```

Weak - evitar reference cycles:

```
use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    value: i32,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

fn main() {
    let leaf = Rc::new(Node {
        value: 3,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![]),
    });

    println!(
        "leaf strong = {}, weak = {}",
        Rc::strong_count(&leaf),
        Rc::weak_count(&leaf),
    );

    {
        let branch = Rc::new(Node {
            value: 5,
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(vec![Rc::clone(&leaf)]),
        });

        *leaf.parent.borrow_mut() = Rc::downgrade(&branch);

        println!(
            "branch strong = {}, weak = {}",
            Rc::strong_count(&branch),
            Rc::weak_count(&branch),
        );

        println!(
```

```
            "leaf strong = {}, weak = {}",
            Rc::strong_count(&leaf),
            Rc::weak_count(&leaf),
        );
    }

    println!("leaf parent = {:?}", leaf.parent.borrow().upgrade());
    println!(
        "leaf strong = {}, weak = {}",
        Rc::strong_count(&leaf),
        Rc::weak_count(&leaf),
    );
}
```

NIVEL AVANZADO

22. Concurrencia

Threads:

```
use std::thread;
use std::time::Duration;

fn main() {
    let handle = thread::spawn(|| {
        for i in 1..10 {
            println!("hi number {} from the spawned thread!", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("hi number {} from the main thread!", i);
        thread::sleep(Duration::from_millis(1));
    }

    handle.join().unwrap(); // Esperar a que termine el thread
}
```

move closures:

```
use std::thread;

fn main() {
    let v = vec![1, 2, 3];
```

```
    let handle = thread::spawn(move || {
        println!("Here's a vector: {:?}", v);
    });

    handle.join().unwrap();
}
```

Message passing con channels:

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let val = String::from("hi");
        tx.send(val).unwrap();
        // println!("val is {}", val); // Error! val fue movido
    });

    let received = rx.recv().unwrap();
    println!("Got: {}", received);
}

// Múltiples valores
fn multiple_values() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let vals = vec![
            String::from("hi"),
            String::from("from"),
            String::from("the"),
            String::from("thread"),
        ];

        for val in vals {
            tx.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });

    for received in rx {
        println!("Got: {}", received);
    }
}
```

```rust
// Múltiples productores
fn multiple_producers() {
    let (tx, rx) = mpsc::channel();

    let tx1 = tx.clone();
    thread::spawn(move || {
        let vals = vec![
            String::from("hi"),
            String::from("from"),
            String::from("the"),
            String::from("thread"),
        ];

        for val in vals {
            tx1.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });

    thread::spawn(move || {
        let vals = vec![
            String::from("more"),
            String::from("messages"),
            String::from("for"),
            String::from("you"),
        ];

        for val in vals {
            tx.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });

    for received in rx {
        println!("Got: {}", received);
    }
}
```

Shared state con Mutex:

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];
```

```
    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();
            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("Result: {}", *counter.lock().unwrap());
}
```

Send y Sync traits:

```
// Send: el tipo puede transferir ownership entre threads
// Sync: el tipo es seguro para referenciar desde múltiples threads

use std::rc::Rc;
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // Rc<T> NO es Send
    let data = Rc::new(5);
    // let handle = thread::spawn(move || {
    //     println!("{}", data); // Error!
    // });

    // Arc<T> SÍ es Send
    let data = Arc::new(5);
    let handle = thread::spawn(move || {
        println!("{}", data);
    });

    handle.join().unwrap();

    // Implementar Send y Sync manualmente es unsafe
    // La mayoría de tipos automáticamente los implementan si es seguro
}
```

Deadlock prevention:

```
use std::sync::{Arc, Mutex};
```

```rust
use std::thread;

fn main() {
    let data1 = Arc::new(Mutex::new(0));
    let data2 = Arc::new(Mutex::new(0));

    let data1_clone = Arc::clone(&data1);
    let data2_clone = Arc::clone(&data2);

    let handle1 = thread::spawn(move || {
        let _lock1 = data1_clone.lock().unwrap();
        println!("Thread 1 got lock1");

        let _lock2 = data2_clone.lock().unwrap();
        println!("Thread 1 got lock2");
    });

    let handle2 = thread::spawn(move || {
        let _lock2 = data2.lock().unwrap();
        println!("Thread 2 got lock2");

        let _lock1 = data1.lock().unwrap();
        println!("Thread 2 got lock1");
    });

    handle1.join().unwrap();
    handle2.join().unwrap();
}
```

23. Programación Asíncrona

async/await básico:

```rust
use futures::executor::block_on;

async fn hello_world() {
    println!("hello, world!");
}

async fn say_hello() {
    hello_world().await;
}

fn main() {
    let future = hello_world();
    block_on(future);
```

```
    block_on(say_hello());
}
```

Async functions y .await:

```
async fn learn_song() -> String {
    "song lyrics".to_string()
}

async fn sing_song(song: String) {
    println!("Singing: {}", song);
}

async fn dance() {
    println!("Dancing!");
}

async fn learn_and_sing() {
    let song = learn_song().await;
    sing_song(song).await;
}

async fn async_main() {
    let f1 = learn_and_sing();
    let f2 = dance();

    // join! ejecuta ambos concurrentemente
    futures::join!(f1, f2);
}

fn main() {
    block_on(async_main());
}
```

Streams:

```
use futures::stream::{self, StreamExt};

async fn sum_with_next(mut stream: Pin<Box<dyn Stream<Item = i32>>>) -> i32 {
    let mut sum = 0;
    while let Some(item) = stream.next().await {
        sum += item;
    }
    sum
}

async fn main() {
```

```
    let stream = stream::iter(vec![1, 2, 3, 4, 5]).boxed();
    let sum = sum_with_next(stream).await;
    println!("Sum: {}", sum);
}
```

Select - ejecutar múltiples futures:

```
use futures::{
    future::FutureExt,
    pin_mut,
    select,
};

async fn task_one() { /* ... */ }
async fn task_two() { /* ... */ }

async fn race_tasks() {
    let t1 = task_one().fuse();
    let t2 = task_two().fuse();

    pin_mut!(t1, t2);

    select! {
        () = t1 => println!("task one completed first"),
        () = t2 => println!("task two completed first"),
    }
}
```

Tokio runtime:

```
// Cargo.toml:
// [dependencies]
// tokio = { version = "1", features = ["full"] }

use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    println!("Hello, world!");

    let handle = tokio::spawn(async {
        sleep(Duration::from_millis(100)).await;
        println!("Task finished");
    });

    handle.await.unwrap();
}
```

Async channels:

```rust
use tokio::sync::{mpsc, oneshot};

#[tokio::main]
async fn main() {
    // mpsc channel
    let (tx, mut rx) = mpsc::channel(32);

    tokio::spawn(async move {
        for i in 0..10 {
            if tx.send(i).await.is_err() {
                break;
            }
        }
    });

    while let Some(i) = rx.recv().await {
        println!("got = {}", i);
    }

    // oneshot channel
    let (tx, rx) = oneshot::channel();

    tokio::spawn(async move {
        tx.send("Hello from task").unwrap();
    });

    let msg = rx.await.unwrap();
    println!("{}", msg);
}
```

24. Macros

Macro declarativo con macro_rules!:

```rust
macro_rules! vec {
    ( $( $x:expr ),* ) => {
        {
            let mut temp_vec = Vec::new();
            $(
                temp_vec.push($x);
            )*
            temp_vec
        }
    };
}
```

```rust
// Macro más compleja
macro_rules! create_function {
    ($func_name:ident) => {
        fn $func_name() {
            println!("You called {:?}()", stringify!($func_name));
        }
    };
}

create_function!(foo);
create_function!(bar);

fn main() {
    foo();
    bar();
}
```

Designadores de fragmentos:

```rust
macro_rules! create_function {
    ($func_name:ident) => {
        fn $func_name() {
            println!("You called {:?}()", stringify!($func_name));
        }
    };
}

// item: items como funciones, structs, modules, etc.
// block: un bloque de código
// stmt: una declaración
// pat: un patrón
// expr: una expresión
// ty: un tipo
// ident: un identificador
// path: un path (e.g. foo, ::std::mem::replace, transmute::<_, int>)
// meta: un meta item; contenidos de atributos
// tt: un token tree
// vis: un visibilidad qualifier
```

Repetición en macros:

```rust
macro_rules! find_min {
    ($x:expr) => ($x);
    ($x:expr, $($y:expr),+) => (
        std::cmp::min($x, find_min!($($y),+))
    )
}
```

```rust
fn main() {
    println!("{}", find_min!(1u32));
    println!("{}", find_min!(1u32 + 2, 2u32));
    println!("{}", find_min!(5u32, 2u32 * 3, 4u32));
}
```

println! y format!:

```rust
macro_rules! my_print {
    ($($arg:tt)*) => {
        print!("Debug: ");
        print!($($arg)*);
    };
}

fn main() {
    my_print!("Hello, {}!\n", "world");
}
```

25. Unsafe Rust

Unsafe superpowers:

```rust
fn main() {
    let mut num = 5;

    let r1 = &num as *const i32;       // raw pointer inmutable
    let r2 = &mut num as *mut i32;     // raw pointer mutable

    // Crear raw pointer a dirección arbitraria (peligroso)
    let address = 0x012345usize;
    let r = address as *const i32;

    unsafe {
        println!("r1 is: {}", *r1);
        println!("r2 is: {}", *r2);

        *r2 = 10;
        println!("r2 is now: {}", *r2);
    }
}
```

Funciones unsafe:

```rust
unsafe fn dangerous() {
    println!("This is dangerous!");
}
```

```rust
fn main() {
    unsafe {
        dangerous();
    }
}
```

Crear abstracciones seguras sobre código unsafe:

```rust
use std::slice;

fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();
    let ptr = values.as_mut_ptr();

    assert!(mid <= len);

    unsafe {
        (
            slice::from_raw_parts_mut(ptr, mid),
            slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}

fn main() {
    let mut v = vec![1, 2, 3, 4, 5, 6];
    let (left, right) = split_at_mut(&mut v, 3);
    println!("Left: {:?}, Right: {:?}", left, right);
}
```

extern functions:

```rust
extern "C" {
    fn abs(input: i32) -> i32;
}

fn main() {
    unsafe {
        println!("Absolute value of -3 according to C: {}", abs(-3));
    }
}

// Llamar Rust desde otros lenguajes
#[no_mangle]
pub extern "C" fn call_from_c() {
    println!("Just called a Rust function from C!");
}
```

Variables static mutables:

```
static HELLO_WORLD: &str = "Hello, world!";

static mut COUNTER: usize = 0;

fn add_to_count(inc: usize) {
    unsafe {
        COUNTER += inc;
    }
}

fn main() {
    println!("name is: {}", HELLO_WORLD);

    add_to_count(3);

    unsafe {
        println!("COUNTER: {}", COUNTER);
    }
}
```

Unsafe traits:

```
unsafe trait Foo {
    // métodos van aquí
}

unsafe impl Foo for i32 {
    // implementación va aquí
}

fn main() {}
```

Acceso a uniones (unions):

```
#[repr(C)]
union MyUnion {
    f1: u32,
    f2: f32,
}

fn main() {
    let u = MyUnion { f1: 1 };

    unsafe {
        let f = u.f2;
        println!("f2: {}", f);
```

```
    }
}
```

26. FFI (Foreign Function Interface)

Llamar funciones C desde Rust:

```
// build.rs
extern crate cc;

fn main() {
    cc::Build::new()
        .file("src/double.c")
        .compile("double");
}

// src/double.c
int double_input(int input) {
    return input * 2;
}

// src/lib.rs
extern "C" {
    fn double_input(input: i32) -> i32;
}

pub fn double_rust_wrapper(input: i32) -> i32 {
    unsafe { double_input(input) }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        assert_eq!(double_rust_wrapper(2), 4);
    }
}
```

Exponer funciones Rust a C:

```
// Cargo.toml
[lib]
name = "embed"
crate-type = ["cdylib"]
```

```rust
// src/lib.rs
use std::ffi::{CStr, CString};
use std::os::raw::c_char;

#[no_mangle]
pub extern "C" fn rust_greeting(to: *const c_char) -> *mut c_char {
    let c_str = unsafe { CStr::from_ptr(to) };
    let recipient = match c_str.to_str() {
        Err(_) => "there",
        Ok(string) => string,
    };

    let response = format!("Hello {}!", recipient);
    let c_string = CString::new(response).expect("CString::new failed");
    c_string.into_raw()
}

#[no_mangle]
pub extern "C" fn rust_greeting_free(s: *mut c_char) {
    unsafe {
        if s.is_null() { return; }
        CString::from_raw(s);
    };
}
```

Header C correspondiente:

```c
// greeting.h
char* rust_greeting(const char* to);
void rust_greeting_free(char*);
```

Usar desde C:

```c
// main.c
#include <stdio.h>
#include <stdlib.h>
#include "greeting.h"

int main() {
    char* greeting = rust_greeting("World");
    printf("%s\n", greeting);
    rust_greeting_free(greeting);
    return 0;
}
```

Bindgen para generar bindings automáticamente:

```
// Cargo.toml
[build-dependencies]
```

```
bindgen = "0.59"

// build.rs
extern crate bindgen;

use std::env;
use std::path::PathBuf;

fn main() {
    println!("cargo:rustc-link-lib=bzip2");

    let bindings = bindgen::Builder::default()
        .header("wrapper.h")
        .parse_callbacks(Box::new(bindgen::CargoCallbacks))
        .generate()
        .expect("Unable to generate bindings");

    let out_path = PathBuf::from(env::var("OUT_DIR").unwrap());
    bindings
        .write_to_file(out_path.join("bindings.rs"))
        .expect("Couldn't write bindings!");
}
```

27. Optimización y Performance

Perfilado con cargo flamegraph:

```
# Instalar flamegraph
cargo install flamegraph

# Generar flamegraph
cargo flamegraph --bin mi_programa

# Con optimizaciones
cargo flamegraph --release --bin mi_programa
```

Benchmarking con criterion:

```
// Cargo.toml
[dev-dependencies]
criterion = "0.3"

[[bench]]
name = "my_benchmark"
harness = false

// benches/my_benchmark.rs
```

```rust
use criterion::{black_box, criterion_group, criterion_main, Criterion};

fn fibonacci(n: u64) -> u64 {
    match n {
        0 => 1,
        1 => 1,
        n => fibonacci(n-1) + fibonacci(n-2),
    }
}

fn criterion_benchmark(c: &mut Criterion) {
    c.bench_function("fib 20", |b| b.iter(|| fibonacci(black_box(20))));
}

criterion_group!(benches, criterion_benchmark);
criterion_main!(benches);
```

Optimizaciones de compilación:

```toml
# Cargo.toml
[profile.release]
debug = true              # Símbolos de debug en release
lto = true                # Link Time Optimization
codegen-units = 1         # Mejor optimización, compilación más lenta
panic = 'abort'           # No unwinding, binario más pequeño
opt-level = 3             # Máxima optimización

[profile.release-with-debug]
inherits = "release"
debug = true
strip = false
```

Evitar allocaciones innecesarias:

```rust
// Mal - crea String nueva cada vez
fn process_data_bad(data: &[&str]) -> String {
    let mut result = String::new();
    for item in data {
        result = result + item + " ";  // Reallocación en cada iteración
    }
    result
}

// Mejor - usa push_str
fn process_data_better(data: &[&str]) -> String {
    let mut result = String::new();
    for item in data {
        result.push_str(item);
```

```
        result.push(' ');
    }
    result
}

// Mejor aún - preallocar capacidad
fn process_data_best(data: &[&str]) -> String {
    let capacity = data.iter().map(|s| s.len()).sum::<usize>() + data.len();
    let mut result = String::with_capacity(capacity);
    for item in data {
        result.push_str(item);
        result.push(' ');
    }
    result
}

// Óptimo para casos simples
fn process_data_optimal(data: &[&str]) -> String {
    data.join(" ")
}
```

Zero-cost abstractions:

```
// Los iteradores se optimizan a loops manuales
fn sum_traditional(data: &[i32]) -> i32 {
    let mut sum = 0;
    for i in 0..data.len() {
        sum += data[i];
    }
    sum
}

fn sum_iterator(data: &[i32]) -> i32 {
    data.iter().sum()  // Se compila al mismo código assembly
}

fn sum_with_filter(data: &[i32]) -> i32 {
    data.iter()
        .filter(|&&x| x > 0)
        .sum()
}
```

SIMD con std::arch:

```
#[cfg(target_arch = "x86_64")]
use std::arch::x86_64::*;

#[cfg(target_arch = "x86_64")]
```

```rust
fn add_vectors_simd(a: &[f32], b: &[f32]) -> Vec<f32> {
    assert_eq!(a.len(), b.len());
    assert!(a.len() % 4 == 0);

    let mut result = Vec::with_capacity(a.len());

    unsafe {
        for chunk in 0..a.len() / 4 {
            let va = _mm_load_ps(a.as_ptr().add(chunk * 4));
            let vb = _mm_load_ps(b.as_ptr().add(chunk * 4));
            let vsum = _mm_add_ps(va, vb);

            let mut temp = [0.0f32; 4];
            _mm_store_ps(temp.as_mut_ptr(), vsum);
            result.extend_from_slice(&temp);
        }
    }

    result
}
```

NIVEL EXPERTO

28. Patrones Avanzados

Pattern syntax:

```rust
fn main() {
    let x = 1;

    match x {
        1 => println!("one"),
        2 => println!("two"),
        3 => println!("three"),
        _ => println!("anything"),
    }

    // Matching named variables
    let x = Some(5);
    let y = 10;

    match x {
        Some(50) => println!("Got 50"),
        Some(y) => println!("Matched, y = {:?}", y), // y sombrea la variable exterior
        _ => println!("Default case, x = {:?}", x),
```

```
    }

    println!("at the end: x = {:?}, y = {:?}", x, y);
}
```

Multiple patterns y ranges:

```
fn main() {
    let x = 1;

    match x {
        1 | 2 => println!("one or two"),
        3 => println!("three"),
        _ => println!("anything"),
    }

    let x = 5;

    match x {
        1..=5 => println!("one through five"),
        _ => println!("something else"),
    }

    let x = 'c';

    match x {
        'a'..='j' => println!("early ASCII letter"),
        'k'..='z' => println!("late ASCII letter"),
        _ => println!("something else"),
    }
}
```

Destructuring:

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 0, y: 7 };

    let Point { x: a, y: b } = p;
    assert_eq!(0, a);
    assert_eq!(7, b);

    let Point { x, y } = p;
    assert_eq!(0, x);
```

```
    assert_eq!(7, y);

    match p {
        Point { x, y: 0 } => println!("On the x axis at {}", x),
        Point { x: 0, y } => println!("On the y axis at {}", y),
        Point { x, y } => println!("On neither axis: ({}, {})", x, y),
    }
}
```

Guards:

```
fn main() {
    let num = Some(4);

    match num {
        Some(x) if x < 5 => println!("less than five: {}", x),
        Some(x) => println!("{}", x),
        None => (),
    }

    let x = Some(5);
    let y = 10;

    match x {
        Some(50) => println!("Got 50"),
        Some(n) if n == y => println!("Matched, n = {}", n),
        _ => println!("Default case, x = {:?}", x),
    }
}
```

@ bindings:

```
enum Message {
    Hello { id: i32 },
}

fn main() {
    let msg = Message::Hello { id: 5 };

    match msg {
        Message::Hello { id: id_variable @ 3..=7 } => {
            println!("Found an id in range: {}", id_variable)
        },
        Message::Hello { id: 10..=12 } => {
            println!("Found an id in another range")
        },
        Message::Hello { id } => {
            println!("Found some other id: {}", id)
```

```
        },
    }
}
```

29. Procedural Macros

Function-like macros:

```
// Cargo.toml
[lib]
proc-macro = true

[dependencies]
syn = "1.0"
quote = "1.0"
proc-macro2 = "1.0"

// src/lib.rs
extern crate proc_macro;

use proc_macro::TokenStream;
use quote::quote;
use syn;

#[proc_macro]
pub fn make_answer(_item: TokenStream) -> TokenStream {
    "fn answer() -> u32 { 42 }".parse().unwrap()
}
```

Derive macros:

```
use proc_macro::TokenStream;
use quote::quote;
use syn::{parse_macro_input, DeriveInput};

#[proc_macro_derive(HelloMacro)]
pub fn hello_macro_derive(input: TokenStream) -> TokenStream {
    let ast = parse_macro_input!(input as DeriveInput);

    let name = &ast.ident;
    let gen = quote! {
        impl HelloMacro for #name {
            fn hello_macro() {
                println!("Hello, Macro! My name is {}!", stringify!(#name));
            }
        }
    };
```

```
    gen.into()
}

// Usar el macro
use hello_macro::HelloMacro;
use hello_macro_derive::HelloMacro;

#[derive(HelloMacro)]
struct Pancakes;

impl HelloMacro for Pancakes {
    fn hello_macro() {
        println!("Hello, Macro! My name is Pancakes!");
    }
}

fn main() {
    Pancakes::hello_macro();
}
```

Attribute-like macros:

```
#[proc_macro_attribute]
pub fn route(attr: TokenStream, item: TokenStream) -> TokenStream {
    let method = attr.to_string();
    let function = parse_macro_input!(item as syn::ItemFn);
    let fn_name = &function.sig.ident;

    let expanded = quote! {
        #function

        fn register_route() {
            println!("Registering {} route for function {}", #method, stringify!(#fn_name));
        }
    };

    TokenStream::from(expanded)
}

// Uso:
#[route("GET")]
fn index() -> String {
    "Hello, world!".to_string()
}
```

Macro complejo con syn:

```
use proc_macro::TokenStream;
```

```rust
use quote::quote;
use syn::{
    parse_macro_input, Data, DeriveInput, Fields, Index,
};

#[proc_macro_derive(Builder)]
pub fn derive_builder(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    let name = input.ident;
    let builder_name = format!("{}Builder", name);
    let builder_ident = syn::Ident::new(&builder_name, name.span());

    let fields = match input.data {
        Data::Struct(data_struct) => match data_struct.fields {
            Fields::Named(fields_named) => fields_named.named,
            _ => panic!("Builder only supports named fields"),
        },
        _ => panic!("Builder only supports structs"),
    };

    let builder_fields: Vec<_> = fields.iter().map(|f| {
        let name = &f.ident;
        let ty = &f.ty;
        quote! { #name: Option<#ty> }
    }).collect();

    let builder_methods: Vec<_> = fields.iter().map(|f| {
        let name = &f.ident;
        let ty = &f.ty;
        quote! {
            pub fn #name(&mut self, #name: #ty) -> &mut Self {
                self.#name = Some(#name);
                self
            }
        }
    }).collect();

    let build_fields: Vec<_> = fields.iter().map(|f| {
        let name = &f.ident;
        quote! {
            #name: self.#name.take().ok_or(concat!(stringify!(#name), " is required"))?
        }
    }).collect();

    let expanded = quote! {
        pub struct #builder_ident {
```

```
            #(#builder_fields,)*
        }

        impl #builder_ident {
            #(#builder_methods)*

            pub fn build(&mut self) -> Result<#name, Box<dyn std::error::Error>> {
                Ok(#name {
                    #(#build_fields,)*
                })
            }
        }

        impl #name {
            pub fn builder() -> #builder_ident {
                #builder_ident {
                    #(#name: None,)*
                }
            }
        }
    };

    TokenStream::from(expanded)
}
```

30. Embedded Rust

Configuración básica:

```
# Cargo.toml
[package]
name = "app"
version = "0.1.0"
edition = "2021"

[dependencies]
nb = "1.0.0"
cortex-m = "0.7.0"
cortex-m-rt = "0.7.0"
panic-halt = "0.2.0"

# Dependencias específicas del microcontrolador
# stm32f4xx-hal = "0.13.0"

[[bin]]
name = "app"
```

```
test = false
bench = false

[profile.dev]
debug = true
lto = true

[profile.release]
debug = true
lto = true
opt-level = "s"
```

Configuración de memoria:

```
// memory.x
MEMORY
{
  /* NOTE 1 K = 1 KiBi = 1024 bytes */
  FLASH : ORIGIN = 0x08000000, LENGTH = 256K
  RAM : ORIGIN = 0x20000000, LENGTH = 64K
}
```

Programa básico:

```
#![no_std]
#![no_main]

use panic_halt as _; // Estrategia de panic

use cortex_m;
use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    // Código de inicialización aquí

    loop {
        // Loop principal
    }
}
```

GPIO y periféricos:

```
#![no_std]
#![no_main]

use panic_halt as _;

use cortex_m;
```

```rust
use cortex_m_rt::entry;
use stm32f4xx_hal::{
    prelude::*,
    stm32,
    gpio::{gpioa::PA5, Output, PushPull},
};

#[entry]
fn main() -> ! {
    if let (Some(dp), Some(cp)) = (
        stm32::Peripherals::take(),
        cortex_m::peripheral::Peripherals::take(),
    ) {
        let gpioa = dp.GPIOA.split();
        let mut led = gpioa.pa5.into_push_pull_output();

        let rcc = dp.RCC.constrain();
        let clocks = rcc.cfgr.freeze();

        let mut delay = hal::delay::Delay::new(cp.SYST, clocks);

        loop {
            led.set_high();
            delay.delay_ms(500u16);
            led.set_low();
            delay.delay_ms(500u16);
        }
    }

    loop {}
}
```

Manejo de interrupciones:

```rust
#![no_std]
#![no_main]

use panic_halt as _;
use cortex_m;
use cortex_m_rt::entry;
use stm32f4xx_hal::{
    interrupt,
    prelude::*,
    stm32,
    stm32::Interrupt,
};
```

```rust
use core::cell::RefCell;
use cortex_m::interrupt::Mutex;

static COUNTER: Mutex<RefCell<u32>> = Mutex::new(RefCell::new(0));

#[entry]
fn main() -> ! {
    let dp = stm32::Peripherals::take().unwrap();

    // Configurar timer
    let mut timer = dp.TIM2.constrain(&dp.RCC).freeze();
    timer.start(1.hz());
    timer.listen();

    // Habilitar interrupción
    unsafe {
        cortex_m::peripheral::NVIC::unmask(Interrupt::TIM2);
    }

    loop {
        // Loop principal
        cortex_m::asm::wfi(); // Wait for interrupt
    }
}

#[interrupt]
fn TIM2() {
    cortex_m::interrupt::free(|cs| {
        let mut counter = COUNTER.borrow(cs).borrow_mut();
        *counter += 1;
    });

    // Clear interrupt flag
    // timer.clear_interrupt_flag();
}
```

31. WebAssembly

Setup básico:

```toml
# Cargo.toml
[lib]
crate-type = ["cdylib"]

[dependencies]
wasm-bindgen = "0.2"
```

```
[dependencies.web-sys]
version = "0.3"
features = [
  "console",
]
```

Función básica:

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
extern "C" {
    fn alert(s: &str);
}

#[wasm_bindgen]
pub fn greet(name: &str) {
    alert(&format!("Hello, {}!", name));
}

#[wasm_bindgen]
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

Interactuar con DOM:

```
use wasm_bindgen::prelude::*;
use web_sys;

#[wasm_bindgen(start)]
pub fn main() {
    let window = web_sys::window().unwrap();
    let document = window.document().unwrap();
    let body = document.body().unwrap();

    let val = document.create_element("p").unwrap();
    val.set_text_content(Some("Hello from Rust and WebAssembly!"));

    body.append_child(&val).unwrap();
}
```

Game of Life en WebAssembly:

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub struct Universe {
```

```
    width: u32,
    height: u32,
    cells: Vec<Cell>,
}

#[wasm_bindgen]
#[repr(u8)]
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Cell {
    Dead = 0,
    Alive = 1,
}

impl Universe {
    fn get_index(&self, row: u32, column: u32) -> usize {
        (row * self.width + column) as usize
    }

    fn live_neighbor_count(&self, row: u32, column: u32) -> u8 {
        let mut count = 0;
        for delta_row in [self.height - 1, 0, 1].iter().cloned() {
            for delta_col in [self.width - 1, 0, 1].iter().cloned() {
                if delta_row == 0 && delta_col == 0 {
                    continue;
                }

                let neighbor_row = (row + delta_row) % self.height;
                let neighbor_col = (column + delta_col) % self.width;
                let idx = self.get_index(neighbor_row, neighbor_col);
                count += self.cells[idx] as u8;
            }
        }
        count
    }
}

#[wasm_bindgen]
impl Universe {
    pub fn tick(&mut self) {
        let mut next = self.cells.clone();

        for row in 0..self.height {
            for col in 0..self.width {
                let idx = self.get_index(row, col);
                let cell = self.cells[idx];
                let live_neighbors = self.live_neighbor_count(row, col);
```

```
            let next_cell = match (cell, live_neighbors) {
                (Cell::Alive, x) if x < 2 => Cell::Dead,
                (Cell::Alive, 2) | (Cell::Alive, 3) => Cell::Alive,
                (Cell::Alive, x) if x > 3 => Cell::Dead,
                (Cell::Dead, 3) => Cell::Alive,
                (otherwise, _) => otherwise,
            };

            next[idx] = next_cell;
        }
    }

    self.cells = next;
}

pub fn new() -> Universe {
    let width = 64;
    let height = 64;

    let cells = (0..width * height)
        .map(|i| {
            if i % 2 == 0 || i % 7 == 0 {
                Cell::Alive
            } else {
                Cell::Dead
            }
        })
        .collect();

    Universe {
        width,
        height,
        cells,
    }
}

pub fn width(&self) -> u32 {
    self.width
}

pub fn height(&self) -> u32 {
    self.height
}

pub fn cells(&self) -> *const Cell {
```

```
        self.cells.as_ptr()
    }
}
```

Build y uso:

```
# Instalar wasm-pack
curl https://rustwasm.github.io/wasm-pack/installer/init.sh -sSf | sh

# Build
wasm-pack build

# En JavaScript:
import init, { greet, Universe } from "./pkg/hello_wasm.js";

async function run() {
    await init();

    greet("WebAssembly");

    const universe = Universe.new();
    universe.tick();
}

run();
```

32. Ecosistema y Crates Importantes

Desarrollo Web:

```
// Actix Web
use actix_web::{web, App, HttpResponse, HttpServer, Result};

async fn greet(name: web::Path<String>) -> Result<HttpResponse> {
    Ok(HttpResponse::Ok().json(format!("Hello {}!", name)))
}

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        App::new()
            .route("/{name}", web::get().to(greet))
    })
    .bind("127.0.0.1:8080")?
    .run()
    .await
}
```

```rust
// Warp
use warp::Filter;

#[tokio::main]
async fn main() {
    let hello = warp::path!("hello" / String)
        .map(|name| format!("Hello, {}!", name));

    warp::serve(hello)
        .run(([127, 0, 0, 1], 3030))
        .await;
}

// Axum
use axum::{
    response::Json,
    routing::get,
    Router,
};
use serde::Serialize;

#[derive(Serialize)]
struct Message {
    message: String,
}

async fn hello_world() -> Json<Message> {
    Json(Message {
        message: "Hello, World!".to_string(),
    })
}

#[tokio::main]
async fn main() {
    let app = Router::new().route("/", get(hello_world));

    axum::Server::bind(&"0.0.0.0:3000".parse().unwrap())
        .serve(app.into_make_service())
        .await
        .unwrap();
}
```

Serialización:

```rust
use serde::{Deserialize, Serialize};
```

```rust
#[derive(Serialize, Deserialize)]
struct Person {
    name: String,
    age: u8,
    phones: Vec<String>,
}

fn main() {
    let person = Person {
        name: "John Doe".to_string(),
        age: 43,
        phones: vec!["555-1234".to_string(), "555-5678".to_string()],
    };

    // JSON
    let json = serde_json::to_string(&person).unwrap();
    println!("JSON: {}", json);

    let deserialized: Person = serde_json::from_str(&json).unwrap();
    println!("Deserialized: {} is {} years old", deserialized.name, deserialized.age);

    // YAML
    let yaml = serde_yaml::to_string(&person).unwrap();
    println!("YAML: {}", yaml);

    // TOML
    let toml = toml::to_string(&person).unwrap();
    println!("TOML: {}", toml);
}
```

Bases de datos:

```rust
// SQLx (async)
use sqlx::{Connection, SqliteConnection, Row};

#[tokio::main]
async fn main() -> Result<(), sqlx::Error> {
    let mut conn = SqliteConnection::connect("sqlite::memory:").await?;

    sqlx::query("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT NOT NULL)")
        .execute(&mut conn)
        .await?;

    sqlx::query("INSERT INTO users (name) VALUES (?)")
        .bind("Alice")
        .execute(&mut conn)
        .await?;
```

```
    let row = sqlx::query("SELECT id, name FROM users WHERE name = ?")
        .bind("Alice")
        .fetch_one(&mut conn)
        .await?;

    let id: i64 = row.get("id");
    let name: String = row.get("name");

    println!("User {}: {}", id, name);

    Ok(())
}

// Diesel (sync ORM)
#[macro_use]
extern crate diesel;

use diesel::prelude::*;
use diesel::sqlite::SqliteConnection;

#[derive(Queryable)]
struct Post {
    id: i32,
    title: String,
    body: String,
    published: bool,
}

table! {
    posts (id) {
        id -> Integer,
        title -> Text,
        body -> Text,
        published -> Bool,
    }
}

fn main() {
    let connection = SqliteConnection::establish("test.db")
        .expect("Error connecting to database");

    let results = posts::table
        .filter(posts::published.eq(true))
        .limit(5)
        .load::<Post>(&connection)
```

```
        .expect("Error loading posts");

    for post in results {
        println!("{}: {}", post.id, post.title);
    }
}
```

CLI Applications:

```
use clap::{App, Arg, SubCommand};
use std::fs;

fn main() {
    let matches = App::new("My Program")
        .version("1.0")
        .author("Your Name <you@example.com>")
        .about("Does awesome things")
        .arg(Arg::with_name("config")
             .short("c")
             .long("config")
             .value_name("FILE")
             .help("Sets a custom config file")
             .takes_value(true))
        .arg(Arg::with_name("INPUT")
             .help("Sets the input file to use")
             .required(true)
             .index(1))
        .arg(Arg::with_name("v")
             .short("v")
             .multiple(true)
             .help("Sets the level of verbosity"))
        .subcommand(SubCommand::with_name("test")
                    .about("controls testing features")
                    .version("1.3")
                    .author("Someone E. <someone_else@other.com>")
                    .arg(Arg::with_name("debug")
                        .short("d")
                        .help("print debug information verbosely")))
        .get_matches();

    let config = matches.value_of("config").unwrap_or("default.conf");
    println!("Value for config: {}", config);

    let input = matches.value_of("INPUT").unwrap();
    println!("Using input file: {}", input);

    match matches.occurrences_of("v") {
```

```
        0 => println!("No verbose info"),
        1 => println!("Some verbose info"),
        2 => println!("Tons of verbose info"),
        3 | _ => println!("Don't be crazy"),
    }

    if let Some(matches) = matches.subcommand_matches("test") {
        if matches.is_present("debug") {
            println!("Printing debug info...");
        } else {
            println!("Printing normally...");
        }
    }
}
```

Crates útiles por categoría:

Desarrollo general: - `anyhow` - Manejo de errores simplificado - `thiserror` - Derive macro para errores customizados
- `log` + `env_logger` - Logging - `chrono` - Manejo de fechas y tiempo - `uuid` - Generación de UUIDs - `rand` - Números aleatorios - `regex` - Expresiones regulares

Async/Concurrency: - `tokio` - Runtime async más popular - `async-std` - Runtime async alternativo - `rayon` - Paralelización de datos - `crossbeam` - Herramientas de concurrencia - `futures` - Traits y utilidades async

Networking: - `reqwest` - Cliente HTTP - `hyper` - HTTP bajo nivel - `tonic` - gRPC client/server - `quinn` - QUIC implementation

Parsing: - `nom` - Parser combinator - `pest` - Parser generator - `syn` - Parsing de código Rust - `roxmltree` - Parser XML rápido

Gaming: - `bevy` - Game engine moderno - `winit` - Ventanas cross-platform - `wgpu` - Graphics API moderno - `nalgebra` - Álgebra lineal

GUI: - `egui` - GUI inmediato - `iced` - GUI inspirado en Elm - `tauri` - Apps desktop con web tech - `druid` - GUI orientado a datos

Crypto: - `ring` - Cryptografía - `rustls` - TLS implementation - `ed25519-dalek` - Firmas digitales - `sha2` - Hash functions

---

Recursos Adicionales

Documentación oficial: - The Rust Book - Rust by Example - The Rustonomicon (unsafe Rust) - API Documentation

Herramientas esenciales:

```
# Formatear código
cargo fmt

# Linter
cargo clippy

# Documentación
cargo doc --open

# Expansión de macros
cargo expand

# Audit de seguridad
cargo audit

# Benchmarking
cargo bench

# Generar flamegraphs
cargo flamegraph
```

Comandos útiles de Cargo:

```
# Crear workspace
cargo new --lib mylib
cargo new --bin myapp

# Añadir dependencias
cargo add serde
cargo add tokio --features full

# Actualizar dependencias
cargo update

# Verificar sin compilar
cargo check

# Compilar sin ejecutar
cargo build --release

# Ejecutar ejemplos
cargo run --example myexample

# Tests con output
cargo test -- --nocapture

# Tests específicos
```

```
cargo test test_name

# Tests de documentación
cargo test --doc
```

Esta guía cubre desde los conceptos más básicos hasta los temas más avanzados de Rust. La clave está en practicar cada concepto progresivamente, empezando por los fundamentos como ownership y borrowing, que son únicos de Rust y fundamentales para entender todo lo demás.

¿Te gustaría que profundice en algún tema específico o tienes alguna pregunta sobre algún concepto en particular?iterales numéricos let decimal = 98_222; let hex = 0xff; let octal = 0o77; let binary = 0b1111_0000; let byte = b'A'; // solo u8 }

Punto flotante:

```rust
fn main() {
    let x = 2.0;        // f64 (por defecto)
    let y: f32 = 3.0;   // f32
}
```

Booleanos:

```rust
fn main() {
    let t = true;
    let f: bool = false;
}
```

Caracteres:

```rust
fn main() {
    let c = 'z';
    let z = ' ';
    let heart_eyed_cat = ' '; // Unicode scalar values
}
```

Tipos compuestos:

Tuplas:

```rust
fn main() {
    let tup: (i32, f64, u8) = (500, 6.4, 1);

    // Destructuring
    let (x, y, z) = tup;
    println!("El valor de y es: {}", y);

    // Acceso por índice
    let five_hundred = tup.0;
```

```
    let six_point_four = tup.1;
    let one = tup.2;
}
```

Arrays:

```
fn main() {
    let a = [1, 2, 3, 4, 5];            // Inferido
    let b: [i32; 5] = [1, 2, 3, 4, 5]; // Explícito
    let c = [3; 5];                      // [3, 3, 3, 3, 3]

    // Acceso a elementos
    let first = a[0];
    let second = a[1];

    // Arrays son de tamaño fijo conocido en compile time
    println!("La longitud del array es: {}", a.len());
}
```

6. Funciones

Función básica:

```
fn main() {
    println!("¡Hola, mundo!");
    otra_funcion();
}

fn otra_funcion() {
    println!("Otra función.");
}
```

Parámetros:

```
fn main() {
    print_labeled_measurement(5, 'h');
}

fn print_labeled_measurement(value: i32, unit_label: char) {
    println!("La medida es: {}{}", value, unit_label);
}
```

Valores de retorno:

```
fn five() -> i32 {
    5  // Expresión sin punto y coma
}

fn plus_one(x: i32) -> i32 {
```

```
    x + 1  // Return implícito
}

fn main() {
    let x = five();
    let y = plus_one(5);
    println!("El valor de x es: {}", x);
    println!("El valor de y es: {}", y);
}
```

Statements vs Expressions:

```
fn main() {
    let y = {           // Bloque es una expresión
        let x = 3;      // Statement
        x + 1           // Expresión (sin ;)
    };                  // y = 4

    println!("El valor de y es: {}", y);
}
```

7. Control de Flujo

if/else:

```
fn main() {
    let number = 6;

    if number % 4 == 0 {
        println!("number es divisible por 4");
    } else if number % 3 == 0 {
        println!("number es divisible por 3");
    } else if number % 2 == 0 {
        println!("number es divisible por 2");
    } else {
        println!("number no es divisible por 4, 3, o 2");
    }

    // if en let
    let condition = true;
    let number = if condition { 5 } else { 6 };
    println!("El valor de number es: {}", number);
}
```

Loops:

loop infinito:

```
fn main() {
```

```
    let mut counter = 0;

    let result = loop {
        counter += 1;

        if counter == 10 {
            break counter * 2; // Retorna valor
        }
    };

    println!("El resultado es {}", result);
}
```

while:

```
fn main() {
    let mut number = 3;

    while number != 0 {
        println!("{}!", number);
        number -= 1;
    }

    println!("¡DESPEGUE!");
}
```

for:

```
fn main() {
    let a = [10, 20, 30, 40, 50];

    // Iterar sobre elementos
    for element in a {
        println!("El valor es: {}", element);
    }

    // Iterar sobre rango
    for number in (1..4).rev() {
        println!("{}!", number);
    }
    println!("¡DESPEGUE!");

    // Con índices
    for (i, element) in a.iter().enumerate() {
        println!("Índice {}: {}", i, element);
    }
}
```

8. Ownership (Propiedad)

Concepto fundamental de Rust:

Reglas del Ownership: 1. Cada valor tiene una variable que es su owner 2. Solo puede haber un owner a la vez 3. Cuando el owner sale de scope, el valor se elimina

Ejemplo básico:

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;  // s1 se mueve a s2

    // println!("{}", s1); // ¡Error! s1 ya no es válido
    println!("{}", s2);    // OK
}
```

Move vs Copy:

```
fn main() {
    // Tipos que implementan Copy
    let x = 5;
    let y = x;  // x se copia, no se mueve
    println!("x = {}, y = {}", x, y); // Ambos válidos

    // Tipos que no implementan Copy
    let s1 = String::from("hello");
    let s2 = s1.clone();  // Clone explícito
    println!("s1 = {}, s2 = {}", s1, s2); // Ambos válidos
}
```

Ownership y funciones:

```
fn main() {
    let s = String::from("hello");
    takes_ownership(s);  // s se mueve a la función
    // println!("{}", s); // ¡Error! s ya no es válido

    let x = 5;
    makes_copy(x);       // x se copia
    println!("{}", x);   // x sigue siendo válido
}

fn takes_ownership(some_string: String) {
    println!("{}", some_string);
} // some_string sale de scope y se elimina

fn makes_copy(some_integer: i32) {
```

```
    println!("{}", some_integer);
} // some_integer sale de scope, pero como es Copy, no pasa nada
```

Return values y ownership:

```
fn main() {
    let s1 = gives_ownership();         // función mueve valor a s1
    let s2 = String::from("hello");     // s2 entra en scope
    let s3 = takes_and_gives_back(s2);  // s2 se mueve, retorna a s3
} // s3 y s1 salen de scope y se eliminan. s2 ya se movió.

fn gives_ownership() -> String {
    let some_string = String::from("yours");
    some_string  // se retorna y mueve al llamador
}

fn takes_and_gives_back(a_string: String) -> String {
    a_string  // se retorna y mueve al llamador
}
```

NIVEL INTERMEDIO

9. References y Borrowing

Referencias (no toman ownership):

```
fn main() {
    let s1 = String::from("hello");
    let len = calculate_length(&s1);  // Prestamos s1
    println!("La longitud de '{}' es {}.", s1, len); // s1 sigue válido
}

fn calculate_length(s: &String) -> usize {
    s.len()
} // s sale de scope, pero no tiene ownership, así que no pasa nada
```

Referencias mutables:

```
fn main() {
    let mut s = String::from("hello");
    change(&mut s);
    println!("{}", s);
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

Reglas del borrowing:

```rust
fn main() {
    let mut s = String::from("hello");

    // Solo UNA referencia mutable a la vez
    let r1 = &mut s;
    // let r2 = &mut s; // ¡Error!

    println!("{}", r1);

    // O MÚLTIPLES referencias inmutables
    let r1 = &s;
    let r2 = &s;
    println!("{} and {}", r1, r2);

    // Pero no mezclar mutables e inmutables
    let r1 = &s;
    // let r2 = &mut s; // ¡Error!
}
```

Dangling references (evitadas por Rust):

```rust
fn main() {
    // let reference_to_nothing = dangle(); // ¡Error de compilación!
    let string = no_dangle();
}

// fn dangle() -> &String {     // ¡Error!
//     let s = String::from("hello");
//     &s  // Retornamos referencia a valor que será eliminado
// }

fn no_dangle() -> String {
    let s = String::from("hello");
    s  // Movemos el ownership
}
```

10. Slices

String slices:

```rust
fn main() {
    let s = String::from("hello world");

    let hello = &s[0..5];   // "hello"
    let world = &s[6..11];  // "world"
    let slice = &s[..];     // Todo el string
```

```
    let slice2 = &s[6..];   // Desde índice 6 hasta el final

    let word = first_word(&s);
    println!("La primera palabra es: {}", word);
}

fn first_word(s: &String) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}
```

String literals son slices:

```
fn main() {
    let s = "Hello, world!"; // &str (string slice)

    // Mejor práctica: usar &str en lugar de &String
    let word = first_word_improved(s);
    println!("Primera palabra: {}", word);
}

fn first_word_improved(s: &str) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}
```

Array slices:

```
fn main() {
    let a = [1, 2, 3, 4, 5];
    let slice = &a[1..3];  // &[i32]

    assert_eq!(slice, &[2, 3]);
```

```
    for element in slice {
        println!("{}", element);
    }
}
```

11. Structs

Definición básica:

```
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}

fn main() {
    let user1 = User {
        email: String::from("someone@example.com"),
        username: String::from("someusername123"),
        active: true,
        sign_in_count: 1,
    };

    println!("Usuario: {}", user1.username);
}
```

Structs mutables:

```
fn main() {
    let mut user1 = User {
        email: String::from("someone@example.com"),
        username: String::from("someusername123"),
        active: true,
        sign_in_count: 1,
    };

    user1.email = String::from("anotheremail@example.com");
}
```

Función constructora:

```
fn build_user(email: String, username: String) -> User {
    User {
        email,      // Shorthand cuando variable = campo
        username,   // Shorthand
        active: true,
        sign_in_count: 1,
```

```
    }
}
```

Struct update syntax:

```
fn main() {
    let user1 = User {
        email: String::from("someone@example.com"),
        username: String::from("someusername123"),
        active: true,
        sign_in_count: 1,
    };

    let user2 = User {
        email: String::from("another@example.com"),
        ..user1  // Toma el resto de user1
    };

    // user1.username ya no es válido (fue movido)
    // pero user1.active y user1.sign_in_count sí (son Copy)
}
```

Tuple structs:

```
struct Color(i32, i32, i32);
struct Point(i32, i32, i32);

fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0, 0, 0);

    // Color y Point son tipos diferentes aunque tengan la misma estructura
    println!("Color: {}, {}, {}", black.0, black.1, black.2);
}
```

Unit-like structs:

```
struct AlwaysEqual;

fn main() {
    let subject = AlwaysEqual;
}
```

Métodos:

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}
```

```rust
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn width(&self) -> bool {
        self.width > 0
    }

    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!("El área del rectángulo es {} pixeles cuadrados.", rect1.area());
    println!("{:#?}", rect1);
}
```

Funciones asociadas:

```rust
impl Rectangle {
    fn square(size: u32) -> Self {
        Self {
            width: size,
            height: size,
        }
    }
}

fn main() {
    let sq = Rectangle::square(3);  // Llamada con ::
}
```

12. Enums y Pattern Matching

Definición básica:

```rust
enum IpAddrKind {
    V4,
    V6,
```

```
}

fn main() {
    let four = IpAddrKind::V4;
    let six = IpAddrKind::V6;

    route(IpAddrKind::V4);
    route(IpAddrKind::V6);
}

fn route(ip_kind: IpAddrKind) {}
```

Enums con datos:

```
enum IpAddr {
    V4(u8, u8, u8, u8),
    V6(String),
}

enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

fn main() {
    let home = IpAddr::V4(127, 0, 0, 1);
    let loopback = IpAddr::V6(String::from("::1"));

    let m = Message::Write(String::from("hello"));
}
```

Métodos en enums:

```
impl Message {
    fn call(&self) {
        // cuerpo del método
        match self {
            Message::Write(s) => println!("Mensaje: {}", s),
            Message::Move { x, y } => println!("Mover a ({}, {})", x, y),
            _ => println!("Otro mensaje"),
        }
    }
}

fn main() {
    let m = Message::Write(String::from("hello"));
```

```
    m.call();
}
```

Option enum:

```
fn main() {
    let some_number = Some(5);
    let some_char = Some('e');
    let absent_number: Option<i32> = None;

    // No se puede sumar Option<i32> + i32 directamente
    let x: i8 = 5;
    let y: Option<i8> = Some(5);
    // let sum = x + y; // ¡Error!

    let sum = x + y.unwrap_or(0); // Manejo seguro
}
```

match - control de flujo potente:

```
#[derive(Debug)]
enum UsState {
    Alabama,
    Alaska,
    // ... etc
}

enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}

fn value_in_cents(coin: Coin) -> u8 {
    match coin {
        Coin::Penny => {
            println!("Lucky penny!");
            1
        },
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter(state) => {
            println!("State quarter from {:?}!", state);
            25
        },
    }
}
```

```rust
fn main() {
    let coin = Coin::Quarter(UsState::Alaska);
    println!("Valor: {} centavos", value_in_cents(coin));
}
```

Matching con Option:

```rust
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        None => None,
        Some(i) => Some(i + 1),
    }
}

fn main() {
    let five = Some(5);
    let six = plus_one(five);
    let none = plus_one(None);
}
```

Catch-all patterns:

```rust
fn main() {
    let dice_roll = 9;

    match dice_roll {
        3 => add_fancy_hat(),
        7 => remove_fancy_hat(),
        other => move_player(other),  // Captura valor
    }

    match dice_roll {
        3 => add_fancy_hat(),
        7 => remove_fancy_hat(),
        _ => reroll(),  // No captura valor
    }
}

fn add_fancy_hat() {}
fn remove_fancy_hat() {}
fn move_player(num_spaces: u8) {}
fn reroll() {}
```

if let - sintaxis concisa:

```rust
fn main() {
    let config_max = Some(3u8);
```

```rust
    // En lugar de match completo:
    match config_max {
        Some(max) => println!("El máximo es {}", max),
        _ => (),
    }

    // Usar if let:
    if let Some(max) = config_max {
        println!("El máximo es {}", max);
    }

    // Con else
    let coin = Coin::Penny;
    let mut count = 0;
    if let Coin::Quarter(state) = coin {
        println!("State quarter from {:?}!", state);
    } else {
        count += 1;
    }
}
```

13. Manejo de Errores

Result enum:

```rust
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file_result = File::open("hello.txt");

    let greeting_file = match greeting_file_result {
        Ok(file) => file,
        Err(error) => match error.kind() {
            ErrorKind::NotFound => match File::create("hello.txt") {
                Ok(fc) => fc,
                Err(e) => panic!("Problem creating the file: {:?}", e),
            },
            other_error => {
                panic!("Problem opening the file: {:?}", other_error);
            }
        },
    };
}
```

Shortcuts para panic: unwrap y expect:

```rust
use std::fs::File;

fn main() {
    // unwrap: panic si Result es Err
    let greeting_file = File::open("hello.txt").unwrap();

    // expect: panic con mensaje personalizado
    let greeting_file = File::open("hello.txt")
        .expect("hello.txt debería estar incluido en este proyecto");
}
```

Propagating errors con ?:

```rust
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let mut username_file = File::open("hello.txt")?;
    let mut username = String::new();
    username_file.read_to_string(&mut username)?;
    Ok(username)
}

// Versión más corta
fn read_username_from_file_short() -> Result<String, io::Error> {
    let mut username = String::new();
    File::open("hello.txt")?.read_to_string(&mut username)?;
    Ok(username)
}

// Aún más corta
fn read_username_from_file_shortest() -> Result<String, io::Error> {
    std::fs::read_to_string("hello.txt")
}
```

? en main:

```rust
use std::error::Error;
use std::fs::File;

fn main() -> Result<(), Box<dyn Error>> {
    let greeting_file = File::open("hello.txt")?;
    Ok(())
}
```

Crear errores personalizados:

```rust
use std::fmt;
```

```rust
#[derive(Debug)]
struct CustomError {
    message: String,
}

impl fmt::Display for CustomError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "Error personalizado: {}", self.message)
    }
}

impl std::error::Error for CustomError {}

fn risky_function() -> Result<i32, CustomError> {
    Err(CustomError {
        message: String::from("Algo salió mal"),
    })
}

fn main() {
    match risky_function() {
        Ok(value) => println!("Éxito: {}", value),
        Err(e) => println!("Error: {}", e),
    }
}
```

panic! vs Result:

```rust
// Usar panic! cuando:
// - Error no recuperable
// - En ejemplos/prototipos
// - En tests

// Usar Result cuando:
// - Error es esperado y recuperable
// - El llamador puede decidir qué hacer
// - En código de producción

fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err(String::from("División por cero"))
    } else {
        Ok(a / b)
    }
}

fn main() {
```

```
    match divide(10.0, 0.0) {
        Ok(result) => println!("Resultado: {}", result),
        Err(e) => println!("Error: {}", e),
    }
}
```

14. Colecciones

Enteros:

```
fn main() {
    let a: i8 = -128;         // 8-bit con signo
    let b: u8 = 255;          // 8-bit sin signo
    let c: i32 = -2_147_483_648; // 32-bit con signo (por defecto)
    let d: u32 = 4_294_967_295;  // 32-bit sin signo
    let e: isize = -9223372036854775808; // Tamaño de arquitectura

    // Leer elementos
    let third: &i32 = &v[2];  // Puede panic si índice no existe
    println!("El tercer elemento es {}", third);

    let third: Option<&i32> = v.get(2);  // Retorna Option
    match third {
        Some(third) => println!("El tercer elemento es {}", third),
        None => println!("No hay tercer elemento."),
    }

    // Iterar
    for i in &v {
        println!("{}", i);
    }

    // Iterar y modificar
    for i in &mut v {
        *i += 50;
    }

    // Almacenar diferentes tipos con enum
    enum SpreadsheetCell {
        Int(i32),
        Float(f64),
        Text(String),
    }

    let row = vec![
        SpreadsheetCell::Int(3),
```

```
        SpreadsheetCell::Text(String::from("blue")),
        SpreadsheetCell::Float(10.12),
    ];
}
```

HashMap:

```
use std::collections::HashMap;

fn main() {
    // Crear HashMap
    let mut scores = HashMap::new();

    scores.insert(String::from("Blue"), 10);
    scores.insert(String::from("Yellow"), 50);

    // Crear desde vectores
    let teams = vec![String::from("Blue"), String::from("Yellow")];
    let initial_scores = vec![10, 50];
    let mut scores: HashMap<_, _> =
        teams.into_iter().zip(initial_scores.into_iter()).collect();

    // Ownership
    let field_name = String::from("Favorite color");
    let field_value = String::from("Blue");

    let mut map = HashMap::new();
    map.insert(field_name, field_value);
    // field_name y field_value ya no son válidos

    // Acceder valores
    let team_name = String::from("Blue");
    let score = scores.get(&team_name).copied().unwrap_or(0);

    // Iterar
    for (key, value) in &scores {
        println!("{}: {}", key, value);
    }

    // Actualizar valores

    // Sobrescribir
    scores.insert(String::from("Blue"), 25);

    // Solo insertar si no existe
    scores.entry(String::from("Yellow")).or_insert(50);
    scores.entry(String::from("Blue")).or_insert(50);
```

```rust
    // Actualizar basado en valor anterior
    let text = "hello world wonderful world";
    let mut map = HashMap::new();

    for word in text.split_whitespace() {
        let count = map.entry(word).or_insert(0);
        *count += 1;
    }

    println!("{:?}", map);
}
```

String:

```rust
fn main() {
    // Crear strings
    let mut s = String::new();
    let s = "initial contents".to_string();
    let s = String::from("initial contents");

    // Actualizar strings
    let mut s = String::from("foo");
    s.push_str("bar");
    s.push('!');

    // Concatenación
    let s1 = String::from("Hello, ");
    let s2 = String::from("world!");
    let s3 = s1 + &s2; // s1 se mueve y ya no es válido

    // format! macro
    let s1 = String::from("tic");
    let s2 = String::from("tac");
    let s3 = String::from("toe");
    let s = format!("{}-{}-{}", s1, s2, s3); // No toma ownership

    // Indexing strings (¡No se puede!)
    let s1 = String::from("hello");
    // let h = s1[0]; // ¡Error!

    // Slicing strings (cuidado con UTF-8)
    let hello = "      ";
    let s = &hello[0..4]; // " " (cada carácter cirílico son 2 bytes)

    // Iterar sobre strings
    for c in "  ".chars() {
```

```
        println!("{}", c);
    }

    for b in "  ".bytes() {
        println!("{}", b);
    }
}
```

Vector (Vec): “‘rust fn main() { // Crear vectors let v: Vec = Vec::new(); let mut v = vec![1, 2, 3]; // macro vec!

```
// Añadir elementos
v.push(5);
v.push(6);
v.push(7);
v.push(8);

// L
```